

Reviewer Pack — Cycle-Space Cocycle Imbalance Lower-Bounds Resolution Width ...

Entry 09c50eb1a365 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 03:18:35 UTC

Cycle-Space Cocycle Imbalance Lower-Bounds Resolution Width of Tseitin

Entry ID: 09c50eb1a365 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 03:18:35 UTC

1. Conjecture

Field A (mathematical branch): Algebraic graph theory: GF(2) cycle space and edge-boundary cocycle imbalance of expander graphs **Field B** (complexity object): Resolution refutation width of Tseitin formulas on bounded-degree expanders

Statement:

Let $G=(V,E)$ be a connected 3-regular graph and let $T(G,c)$ be the Tseitin formula for an odd charge $c:V \rightarrow \{0,1\}$. Define the cocycle imbalance $I(G) = \min$ over nonempty S subset V of $|E(S, V^c)| / \min(|S|, |V^c|)$, and let $w(T(G,c))$ denote the minimum width of any resolution refutation of $T(G,c)$. The conjecture is the linear lower bound $w(T(G,c)) \geq \lfloor I(G) * \min(|S|, |V^c|) / 2 \rfloor + 1$, where S is the cocycle-imbalance minimizer; equivalently, resolution width is at least half the minimum odd-charge edge-boundary plus one.

Rationale (proposer's reasoning):

Ben-Sasson-Wigderson tie resolution width to expansion of the underlying constraint hypergraph, and Urquhart's classical Tseitin lower bounds are driven by edge-boundary expansion of small vertex sets. Reformulating expansion as a GF(2) cocycle imbalance (the dual of cycle-space sparsity) gives a coordinate-free invariant that should sharpen the constant in width lower bounds, and falsification on a single 3-regular graph would refute a clean algebraic strengthening of the BSW bound.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a8f0315a35b34080

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 30 random connected 3-regular graphs per n in $\{8,10,\dots,20\}$ (≥ 210 instances total), each with a random odd Tseitin charge, compute $I(G)$ by brute force and $w(T(G,c))$

via width-bounded resolution. Conjecture is SUPPORTED iff every instance satisfies $w \geq \lfloor \text{floor}(I(G) \min(|S|, |V \setminus S|)/2) + 1 \rfloor$; FALSIFIED by any single violating instance.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.97	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.97	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin formula resolution width expander edge expansion lower bound - resolution width Tseitin bounded-degree expander cycle space GF(2) - Tseitin tautology refutation width cocycle edge boundary 3-regular

Top relevant hits considered: - [http://arxiv.org/abs/1411.3958v3] Edge-state enhanced transport in a 2-dimensional quantum walk - [http://arxiv.org/abs/2209.05839v3] On bounded depth proofs for Tseitin formulas on the grid; revisited - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/0804.4433v1] SPACE: the SPECTROSCOPIC All-sky Cosmic Explorer - [http://arxiv.org/abs/2405.02292v1] ALOHA 2: An Enhanced Low-Cost Hardware for Bimanual Teleoperation - [http://arxiv.org/abs/1303.4033v1] Two-dimensional magnetic interactions in LaFeAsO - [http://arxiv.org/abs/2404.04038v1] Refutability as Recursive as Provability

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=180.1s

5.1 Generated Python source

```
import random
from itertools import combinations

def generate_3_regular_graph(n):
    if n % 2 != 0:
        raise ValueError("n must be even for a 3-regular graph")

    G = {i: {} for i in range(n)}
    degree = 3

    while True:
        edges = set()
        for u in range(n):
            available_neighbors = [v for v in range(n) if v != u and v not in G[u]]
            neighbors = random.sample(available_neighbors, degree)
            for v in neighbors:
                edges.add((min(u, v), max(u, v)))
```

```

        if len(edges) == n * degree // 2:
            break

    for (u, v) in edges:
        G[u][v] = 1
        G[v][u] = 1

    return G

def compute_cocycle_imbalance(G):
    n = len(G)
    min_bound = float('inf')

    for S in range(1, n):
        bound = abs(sum(G[u][v] for u, v in combinations(S, 2))) / min(len(S), n - len(S))
        if bound < min_bound:
            min_bound = bound

    return min_bound

def resolve_clause_set(clause_set, width):
    stack = []
    while clause_set:
        new_clause_set = set()
        for clause in clause_set:
            if any(lit not in stack and -lit not in stack for lit in clause):
                stack.append(random.choice(clause))
            else:
                new_clause_set.add(tuple(sorted(lit for lit in clause if lit not in stack and -lit
                clause_set = new_clause_set
            if len(stack) > width:
                return False
    return True

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [8, 10, 12, 14, 16, 18, 20]
    results = []

    for n in n_values:
        G = generate_3_regular_graph(n)
        c = {i: random.choice([0, 1]) for i in range(n)}

        I_G = compute_cocycle_imbalance(G)
        min_bound = float('inf')
        S_star = None

        for S in range(1, n):
            bound = abs(sum(G[u][v] for u, v in combinations(S, 2))) / min(len(S), n - len(S))
            if bound < min_bound:
                min_bound = bound
                S_star = S

        w_T_G_c = 0
        while not resolve_clause_set({(c[u] ^ c[v]) * (1 - G[u][v]) for u, v in G}, w_T_G_c):

```

```

        w_T_G_c += 1

    results.append({
        "n": n,
        "I(G)": I_G,
        "w(T(G,c))": w_T_G_c,
        "S*": S_star
    })

total_instances = len(results)
support_count = sum(1 for result in results if result["w(T(G,c))"] >= result["I(G)"] * min(len(
    result["S*"]), result["I(G)"]))

conjecture_holds = support_count == total_instances
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "resolution_width",
    "metric_value": sum(result["w(T(G,c))"] for result in results) / total_instances,
    "instances_tested": total_instances,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={sum(result['metric_value'] for result in results) / len(results)}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 180s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 180s (returncode=124) without producing any RESULT line, so the pre-registered support condition over ≥ 210 instances was never evaluated. | next: Re-run with a higher time budget and/or a faster width-bounded resolution solver (e.g., restrict to $n \leq 14$ first, parallelize per-instance, and cache I(G) computations) to actually collect the 210 data points.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	16553
2	preregistration	claude_max	opus	0	0	12086
3	novelty	claude_max	opus	0	0	3115
4	novelty	claude_max	opus	0	0	9577
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19611
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12019
7	judge	claude_max	opus	0	0	5644

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 78604 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/09c50eb1a365.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/09c50eb1a365.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/09c50eb1a365.tar.gz` (if generated)